Build Prompt Pack: the Export Incentive and Payment Tracker

Twenty engineered prompts to build the full app: parsing, matching, incentive maths, eBRC export, trial, Razorpay payments with webhooks, onboarding and polished mobile-friendly UI.

- A master rules file plus 19 sequenced build prompts
- Every prompt ends with acceptance criteria you can test
- Razorpay subscriptions with a signed, idempotent webhook engine
- Freemium, a card-free 7-day trial, guided onboarding
- Go-live checklist and three review prompts

Prepared 12 November 2026 | Working document: verify regulatory details before relying on them

How to use this prompt pack

This pack contains **20 prompts** that you paste, in order, into an AI coding tool (Claude Code, Cursor, Windsurf or similar). Each prompt builds one slice of the app and ends with **acceptance criteria** so you can tell whether the AI really finished. The app name below, **ExportLeak**, is a placeholder: rename it before you launch.

The working method

1. Create an empty Git repository. Save **Prompt 0 (Master Context)** as a file called **CLAUDE.md** (or the rules file your tool uses) so the AI re-reads it every session.
2. Paste **one prompt at a time**, in order. Do not skip ahead: later prompts assume earlier tables and components exist.
3. After each prompt: run the app, click through the feature on a phone-sized screen, run the tests, then **commit**. If something is wrong, paste the acceptance criteria back and say which line failed.
4. Use **Razorpay test mode** until Prompt 14 passes completely. Never put live keys in the repository.
5. Before trusting the document parser and the eBRC export, test them on **real documents** from consultants (with permission). Synthetic data proves nothing about real PDFs.

Three things the AI must never invent
1) **Incentive rates** (RoDTEP, drawback): they come from official notifications and are loaded as data, never typed into code. 2) **The bulk eBRC Excel layout**: download the latest sample file from the DGFT portal and give it to the AI as a fixture. 3) **Legal deadlines**: keep them in a configuration table with an effective date, because the FEMA realisation period has changed several times (9 months from 1 Oct 2026 for most exports).

Recommended stack (for a solo student on a tight budget)

Layer	Choice	Why
Web app	Next.js (App Router) + TypeScript + Tailwind CSS + shadcn/ui + Framer Motion	One codebase for desktop web and mobile web; strong AI-tool support.
Database and auth	Supabase (Postgres, Auth, Storage, Row Level Security)	Free tier to start; per-customer data isolation through RLS.
Payments	Razorpay Subscriptions + Standard Checkout + webhooks	Your stated choice; supports recurring billing.
Email	Resend (or similar)	Alerts, trial reminders, receipts.
Background jobs	Supabase scheduled functions or Vercel Cron	Daily deadline and expiry checks.
Document parsing	PDF text extraction first, AI extraction with schema validation second	Cheap and accurate; AI used only where needed.
Excel	exceljs or SheetJS	Read IRM sheets, write the bulk eBRC file.
Errors and uptime	Sentry free tier + an uptime monitor	You will not be watching the app all day.

Check current pricing and terms for every service. For example, some free hosting tiers restrict commercial use. Your budget is under ₹3,000 per month for tools and hosting, so cap AI usage per document (Prompt 6).

Plans and trial this pack builds (you can change the numbers later in one config file)

Plan	Price	Limits (tunable)	Notes
Free	₹0 forever	1 IEC, 5 shipping bills per month, dashboard view, no eBRC export	Zero-friction signup; shows the leak.
7-day Pro trial	₹0, no card	Exporter-plan features	Starts automatically after the first upload; reverts to Free; no payment details needed.
Exporter	₹999 per month or annual price	1 IEC, 50 shipping bills per month, eBRC export, alerts	Annual plan at a discount.
Consultant	₹2,999 per month or annual price	Up to 25 IECs, pooled monthly shipping-bill allowance, team seats	The main plan to sell.

Contents

#	Prompt	#	Prompt
0	Master Context (save as CLAUDE.md)	10	Bulk eBRC Excel generator
1	Project scaffold and design system	11	Plans, entitlements, usage metering
2	Database schema and security rules	12	Trial engine (7 days, no card)
3	Authentication, organizations, roles	13	Razorpay checkout and subscriptions
4	Onboarding wizard and demo workspace	14	Razorpay webhooks (activation engine)
5	Document upload and storage	15	Billing UI, pricing, legal pages
6	Shipping Bill parser and review screen	16	Alerts and notifications
7	IRM import and matching engine	17	Landing page, free tools, SEO
8	Incentive calculation engine	18	Mobile, motion, accessibility polish
9	Leakage dashboard and deadlines	19	Security, privacy, testing, deployment
		+	Appendices: go-live checklist, review prompts, your to-do list

Prompt 0: Master Context (save as CLAUDE.md)

Paste this at the start of the project and keep it as a permanent rules file.

PROMPT 0 - MASTER CONTEXT

```
You are the senior full-stack engineer on "ExportLeak" (placeholder name), a web app for small Indian
exporters and the EXIM/DGFT consultants who serve them.

PRODUCT
Users upload Shipping Bill PDFs, a bank Inward Remittance Message (IRM) export, and a customs
incentive scroll or ledger export. The app (1) extracts shipment data, (2) matches each shipping bill
to its payment, (3) calculates expected RoDTEP and duty drawback and compares with what was credited,
(4) flags unclaimed money, expiring scrips and approaching realisation deadlines, (5) generates a bulk
eBRC upload Excel in the exact format DGFT publishes, and (6) exports a reconciliation report (PDF and
Excel).

USERS
- Exporter: owner or accounts person, 1 IEC, often non-technical, often on a phone.
- Consultant: manages up to 25 exporter IECs, power user, mostly desktop.

HARD RULES (never break)
```

1. NEVER ask for, accept or store DSC tokens, portal usernames or passwords. NEVER scrape or automate government portals. Users download files themselves and upload them.
2. Money is stored as integers in paise (INR) or minor units (foreign currency). Never use floating point for money. Display INR in Indian grouping (₹10,00,000) with en-IN formatting.
3. Incentive rates, caps, scrip validity and the FEMA realisation period live in database tables with effective_from / effective_to dates. Never hard-code them. Never invent rates.
4. The bulk eBRC Excel must match the official DGFT sample file, which lives in /fixtures/dgft-bulk-ebrc-sample.xlsx. If a column or format rule is unknown, STOP and ask me instead of guessing.
5. Every data table has org_id/iec_id and Postgres Row Level Security. A user can only ever read rows of organizations they belong to. Write tests for this.
6. Every extracted field carries a confidence score and a source reference (page number). Low-confidence fields require human confirmation before they affect calculations.
7. Everything user-visible is mobile-first (360px width up). Tables become card lists on small screens. Tap targets >= 44px. Respect prefers-reduced-motion.
8. Timezone Asia/Kolkata. Dates shown as DD MMM YYYY. Currency amounts show the currency code.
9. Plan limits and prices come from one config module (config/plans.ts) and are enforced on the SERVER, never only in the UI.
10. Payments: Razorpay Subscriptions. The webhook is the source of truth for activating or deactivating a plan. Never trust the browser's payment success callback for activation.
11. Write an audit_log row for every create/update/delete of business data and every billing state change.
12. No TODO comments, no placeholder data in production paths, no console.log of personal or financial data.

ENGINEERING STYLE
Next.js App Router, TypeScript strict, Tailwind, shadcn/ui, Framer Motion, Zod validation at every boundary, server actions or route handlers with auth checks, Vitest unit tests, Playwright end-to-end tests for critical flows. Small files, clear names, comments only where intent is non-obvious.

HOW TO WORK
For each task I give you: restate the goal in 3 lines, list the files you will create or change, implement, run type-check, lint and tests, then summarise what you did and what you could not verify. If my instruction conflicts with a hard rule above, tell me and follow the rule.

Phase A: Foundation

Prompt 1: Project scaffold and design system

PROMPT 1 - SCAFFOLD AND DESIGN SYSTEM

GOAL
Create the project skeleton and a reusable design system so every later screen looks consistent, professional and works on mobile and desktop.

DO
1. Scaffold Next.js (App Router, TypeScript strict), Tailwind, shadcn/ui, Framer Motion, ESLint, Prettier, Vitest, Playwright. Add scripts: dev, build, lint, typecheck, test, e2e.
2. Create /config/plans.ts (empty typed structure for now), /config/app.ts (app name placeholder, support email, WhatsApp number), and .env.example listing every variable we will need (Supabase URL and keys, Razorpay key id/secret/webhook secret, plan ids, Resend key, app URL). Never commit real secrets.
3. Design tokens: a calm, trustworthy fintech look. Primary deep navy, accent teal, success green, warning amber, danger red, neutral greys. Light and dark mode via CSS variables. Typography: Inter for UI, tabular numbers for money. 4/8px spacing scale, 12px card radius, soft shadows.
4. Build base components: Button (primary, secondary, ghost, destructive, loading state), Card, Badge, Input, Select, Dialog, Sheet (bottom sheet on mobile), Tabs, Toast, Skeleton, EmptyState, Stat (big number with label and trend), ProgressRing, Stepper, DataTable (desktop) that switches to a CardList on screens under 768px, MoneyText (paise to en-IN INR), DateText.
5. App shell: sidebar on desktop, bottom tab bar (4 items + "More") on mobile, top bar with org/IEC switcher, notification bell, user menu. Safe-area padding for phones.
6. Motion rules in one file: page transition 180ms fade/slide, list items stagger 40ms, number count-up for money stats (600ms), skeletons instead of spinners, button press scale 0.98. Everything disabled under prefers-reduced-motion.

```
7. A /design page (dev only) showing every component in light and dark mode, at 360px and 1280px.

ACCEPTANCE CRITERIA
- npm run typecheck, lint, test all pass.
- /design renders without horizontal scroll at 360px.
- Switching dark mode keeps text contrast >= 4.5:1.
- DataTable becomes a card list below 768px (Playwright test with two viewports).
```

Prompt 2: Database schema and security rules

PROMPT 2 - DATABASE SCHEMA AND RLS

```
GOAL
Create the Postgres schema (Supabase migrations) with Row Level Security. Multi-tenant: an
organization can be an exporter or a consultant; a consultant manages many IEC profiles.

TABLES (all with id uuid pk, created_at, updated_at; money in integer minor units)
- organizations(name, type: exporter|consultant, plan_key, plan_status, trial_ends_at,
razorpay_customer_id)
- memberships(user_id, org_id, role: owner|admin|member|viewer)
- iec_profiles(org_id, iec_code, legal_name, gstin, ad_bank_name, default_currency)
- documents(iec_id, kind: shipping_bill|irm_export|scroll|other, storage_path, sha256, original_name,
parse_status: queued|parsing|needs_review|done|failed, uploaded_by)
- shipping_bills(iec_id, document_id, sb_number, sb_date, port_code, leo_date, invoice_no, buyer_name,
currency, fob_minor, exchange_rate, schemes_claimed text[], status, realisation_due_date)
- shipping_bill_items(shipping_bill_id, hs_code, description, quantity, uqc, fob_minor,
field_confidence jsonb)
- remittances(iec_id, irm_number, bank_name, remittance_date, currency, amount_minor, utilised_minor,
raw jsonb)
- sb_remittance_matches(shipping_bill_id, remittance_id, matched_minor, method: auto|manual,
confidence, confirmed_by)
- incentive_rates(scheme: rodtep|drawback|rosctl, hs_code, rate_bps, cap_per_unit_minor, uqc,
effective_from, effective_to, source_note)
- incentive_expectations(shipping_bill_id, scheme, expected_minor, rate_id, calc_detail jsonb)
- incentive_credits(shipping_bill_id, scheme, credited_minor, credited_on, source_document_id)
- scrips(iec_id, scrip_number, scheme, face_minor, generated_on, expires_on, status)
- compliance_rules(key, value jsonb, effective_from, effective_to) -- e.g. realisation_months
- alerts(iec_id, kind, severity, shipping_bill_id, due_on, resolved_at, sent_channels text[])
- subscriptions(org_id, razorpay_subscription_id unique, plan_key, status, current_start,
current_end, charge_at, paid_count, cancel_at_cycle_end, raw jsonb)
- payment_events(id, provider, provider_event_id unique, event_type, payload jsonb, processed_at,
error)
- usage_counters(org_id, period_start, shipping_bills_used)
- audit_log(org_id, actor_id, action, entity, entity_id, diff jsonb)

SECURITY
1. Enable RLS on every table. Policies: a user may SELECT/INSERT/UPDATE rows only where the row's org
(directly or via iec_profiles) is in their memberships. viewer = read-only. payment_events and
subscriptions are written only by the service role.
2. incentive_rates and compliance_rules are readable by all authenticated users, writable only by an
admin flag.
3. Storage bucket "documents" is private; paths are {org_id}/{iec_id}/{uuid}.pdf; storage policies
mirror the table policies.
4. Indexes on every foreign key, on (iec_id, sb_number) unique, on (iec_id, irm_number) unique, and on
alerts(due_on).

ALSO
- Seed script that inserts SYNTHETIC demo data only (clearly fake company names, IEC codes beginning
with "DEMO").
- Generate TypeScript types from the schema.

ACCEPTANCE CRITERIA
- Migrations apply cleanly from empty.
```

```
- An automated test creates two organizations and proves user A can never read or update user B's rows in ANY table.
- Unique constraint stops a duplicate shipping bill upload for the same IEC.
```

Prompt 3: Authentication, organizations and roles

PROMPT 3 - AUTH AND ORGANIZATIONS

```
GOAL
Let people sign up quickly, create an organization (exporter or consultant) and invite team members.

DO
1. Supabase Auth with: Google sign-in and email magic link or OTP (no passwords to forget).
Mobile-friendly login screen with a short trust line: "We never ask for your DSC or portal passwords."
2. After first login: a short form asking "I am an... Exporter / EXIM consultant or CHA / CA firm".
Create the organization (type exporter for the first, consultant for the other two) and an owner
membership.
3. Exporter org: auto-create its single IEC profile after onboarding (Prompt 4). Consultant org: "Add
client IEC" flow with IEC code, legal name, optional GSTIN. IEC code must validate as 10 characters
alphanumeric.
4. Team invites by email with role (admin, member, viewer). Accepting an invite adds a membership.
Owner can remove members.
5. IEC switcher in the top bar for consultants (searchable list, shows each client's open alerts
count).
6. Middleware that protects every /app route, redirects unauthenticated users to /login, and redirects
users without an organization to onboarding.
7. A server helper requireOrgAccess(orgId, minRole) used by every route handler and server action.

EDGE CASES
Invite to an email that already has an account. Last owner trying to leave. User belonging to two
organizations. Expired magic link.

ACCEPTANCE CRITERIA
- Playwright: sign up, create exporter org, land on onboarding. Second user invited as viewer cannot
upload or delete.
- Calling any API route with another org's id returns 403, covered by tests.
```

Prompt 4: Onboarding wizard and demo workspace

PROMPT 4 - GUIDED ONBOARDING

```
GOAL
A guided first-run experience that gets a new user to the "aha moment" (seeing their own money leak,
or a demo of it) in under 5 minutes, and builds trust.

DO
1. Full-screen stepper wizard (works as a sheet on mobile), 4 steps with progress bar, back/next, and
the ability to skip:
   Step 1 "Who are you?" (confirm role, company name, IEC code, GSTIN optional).
   Step 2 "What do you want to check?" (RoDTEP money, payment deadlines, eBRC Excel) as selectable
cards; store as preferences.
   Step 3 "Add your files" explains the 3 files in plain language with a short help text for each and
where to download them (text only; do not include fake portal screenshots). Offers upload now, or "Try
with sample data".
   Step 4 "Your first result" shows the leakage dashboard populated from their upload or from the demo
workspace.
2. "Try with sample data" button creates a SANITISED demo IEC with synthetic shipping bills,
remittances and scroll data so a user (or you, on a sales call) can explore instantly. Demo data is
clearly labelled and deletable in one click.
3. Product tour: after the wizard, 5 coachmark tooltips (dashboard totals, shipments table, alerts,
export button, help). Dismissible, replayable from the Help menu.
4. Persistent onboarding checklist card on the dashboard until complete: Add IEC, Upload shipping
bills, Upload IRM export, Review matches, Download first eBRC file.
```

```
5. Trust panel reachable from the wizard and footer: "We never ask for DSC or passwords", "Your files are private to your organization", "Delete your data anytime", links to Privacy Policy.
6. Start the 7-day Pro trial automatically when the user completes their first upload (Prompt 12 implements the engine; here only call its start function and show a friendly banner).
7. Track onboarding funnel events in a simple events table (step viewed, completed, skipped).

ACCEPTANCE CRITERIA
- A brand-new account reaches a populated dashboard through "Try with sample data" in 3 taps.
- Wizard works fully with keyboard and on a 360px screen.
- Closing the browser mid-wizard resumes at the same step.
```

Phase B: The core engine

Prompt 5: Document upload and storage

PROMPT 5 - DOCUMENT UPLOAD

```
GOAL
Fast, safe, mobile-friendly upload for PDFs, Excel and CSV files.

DO
1. Upload zone: drag-and-drop on desktop, large tap area on mobile with "Choose files" (also allow phone camera/scan to PDF only if feasible; otherwise skip). Multiple files at once, up to 50 PDFs per batch.
2. Client-side checks: type (pdf, xlsx, xls, csv), size limit (e.g. 10 MB per file), duplicate detection by SHA-256 hash within the same IEC.
3. Direct upload to the private Supabase Storage bucket using signed URLs; show per-file progress bars with animated states (queued, uploading, uploaded, parsing, needs review, done, failed) and a retry button.
4. Create a documents row per file and enqueue parsing (Prompt 6). Show a live status list that updates in real time (Supabase Realtime or polling every 3 seconds).
5. Enforce plan limits on the SERVER before accepting the batch (Prompt 11 provides the check): if the monthly shipping-bill allowance would be exceeded, show an upgrade dialog, not a silent failure.
6. Never execute or render uploaded files on the server beyond parsing. Validate magic bytes, not only the extension. Strip PDF JavaScript by never opening in a browser engine.
7. A Documents page: searchable, filterable list (kind, status, date), download original, delete (soft-delete with 30-day purge job).

ACCEPTANCE CRITERIA
- Uploading the same PDF twice shows "Already uploaded" and creates no second row.
- A renamed .exe with a .pdf extension is rejected.
- 50 files upload on a throttled mobile connection without freezing the UI.
```

Prompt 6: Shipping Bill parser and review screen

PROMPT 6 - SHIPPING BILL PARSER

```
GOAL
Turn Shipping Bill PDFs into structured, trustworthy data, with a human review step for anything uncertain.

CONTEXT
I will place 10+ real, anonymised shipping bill PDFs in /fixtures/shipping-bills/. Study their layout before writing the parser. Do not assume a single layout: ICEGATE prints and CHA-generated copies may differ.

DO
1. Pipeline (background job, idempotent, retries with backoff):
   a. Extract text with a PDF text library. If text is empty or very short, mark as scanned and use an OCR fallback (clearly flagged lower confidence).
```

```
   b. Deterministic pass: regex/anchors for shipping bill number, date, port code, LEO date, invoice number, currency, FOB value, and per-item HS code (8 digits), description, quantity, UQC, item FOB. Record page and text span for each field.
   c. AI pass ONLY for fields the deterministic pass could not find or that failed validation. Use structured output with a Zod schema. Send only the minimum text needed. Cap AI spend per document (config value) and fall back to "needs review" when the cap is hit.
   d. Validate: HS code is 8 digits, dates parse and are plausible, item FOBs sum to the bill FOB within a small tolerance, currency is a known ISO code, shipping bill number is numeric of expected length. Failed checks lower confidence.
2. Store results in shipping_bills and shipping_bill_items with field_confidence. Status becomes needs_review if any required field has confidence below a threshold.
3. Review screen: split view on desktop (PDF page on the left with the highlighted source text, editable fields on the right). On mobile: a stacked layout with a "Show source" bottom sheet. Show only the uncertain fields first. One-tap "Confirm all", keyboard shortcuts on desktop (Enter to confirm field, Tab to next).
4. Corrections are saved and written to audit_log. Keep a table of past corrections to improve prompts and regex later.
5. A batch summary after upload: "42 parsed, 5 need review, 1 failed" with direct links.
6. Privacy: document text sent to an AI provider must be disclosed in the Privacy Policy; add a setting flag to disable the AI pass per organization (then everything uncertain goes to review).

ACCEPTANCE CRITERIA
- On the fixtures folder, field-level accuracy report is generated by a test script (precision and recall per field) and committed as a markdown file. Required fields must be >= 95% correct or flagged for review.
- No silent guesses: a low-confidence field is never used in calculations until confirmed.
- Parsing one document never costs more than the configured AI cap.
```

Prompt 7: IRM import and matching engine

PROMPT 7 - IRM IMPORT AND MATCHING

```
GOAL
Import the bank payment list (IRM export from DGFT, as Excel or CSV) and match payments to shipping bills with a transparent, explainable algorithm.

CONTEXT
Real sample exports live in /fixtures/irm/. Inspect the actual column names and formats first, then build a column-mapping layer so small format changes do not break imports. Show the user a mapping preview before importing.

DO
1. Importer: read xlsx/csv, detect the header row, map columns (IRM number, bank, remittance date, currency, amount, utilised amount, buyer or remitter name, any invoice reference), normalise dates and amounts to integer minor units. Upsert by (iec_id, irm_number). Report rows that failed with reasons, never skip silently.
2. Matching engine (pure function, heavily unit-tested):
   - Candidate generation: same currency; payment date on or after the shipping bill date.
   - Scoring signals: invoice number found in the remittance reference, amount equality or near-equality (allowing small bank-charge differences), buyer-name similarity, date proximity.
   - Support one-to-one, one payment to many bills (sum), and many payments to one bill (partial payments). Track the unmatched remainder on each side.
   - Output a confidence score and an explanation string for every suggested match ("Amount equal; invoice INV-2231 found in reference").
3. Auto-confirm only matches above a high threshold; everything else goes to a Review Matches screen: left list of unmatched bills, right list of candidate payments with scores and explanations, one-tap accept/reject, bulk accept for high scores, manual link with amount allocation.
4. Rule: payments in different currencies are never clubbed (DGFT allows clubbing only for same currency). Enforce this in the matcher and in the UI.
5. Every confirm/unmatch writes audit_log. Unmatching is always possible.

ACCEPTANCE CRITERIA
- Unit tests cover exact match, partial payments, split payments, currency mismatch, duplicate invoice numbers and near-equal amounts.
```

```
- On the fixtures, a report shows how many matches were auto-confirmed vs reviewed.
- No match can allocate more than the remittance's remaining balance.
```

Prompt 8: Incentive calculation engine

PROMPT 8 - INCENTIVE ENGINE

```
GOAL
Calculate expected RoDTEP and duty drawback per shipping bill item, compare against what customs
credited, and explain every number.

IMPORTANT
Do NOT type any rates into code. Build an admin import that loads rates from an official-notification
CSV that I will provide into incentive_rates, with effective dates. If the CSV format is unclear, ask
me.

DO
1. Rate importer (admin only): CSV upload with columns scheme, hs_code, rate_bps, cap_per_unit, uqc,
effective_from, effective_to, source_note. Dry-run preview, validation (HS codes 8 digits, no
overlapping date ranges per scheme and HS code), then commit.
2. Calculator (pure, unit-tested):
   - For each item pick the rate row valid on the shipping bill date (or the date the scheme requires;
keep the reference date in config).
   - Expected = min(rate% * item FOB, cap_per_unit * quantity) when a cap exists; else rate% * FOB.
Integer math in minor units with explicit rounding rule documented in code.
   - Return a calc_detail JSON: rate used, cap used, which limb of the formula applied, warnings (unit
mismatch with the rate's UQC, missing rate, scheme not claimed on the shipping bill).
3. Credits importer: parse the customs scroll or ledger export (fixtures in /fixtures/scroll/) into
incentive_credits; map by shipping bill number; same preview and error reporting approach as the IRM
importer.
4. Comparison: for each shipping bill compute expected vs credited per scheme, with a status: ok,
short, missing, not_claimed (scheme was never selected), unknown_rate.
5. Scrips: let users record or import e-scrips (number, face value, generated date); compute expiry
from a validity setting in compliance_rules (default to be confirmed; do not assume) and surface the
next-to-expire scrips.
6. Everything shows its working: tapping an amount opens "How we calculated this" with the formula and
inputs.

ACCEPTANCE CRITERIA
- Table-driven tests with at least 15 cases including cap hit, no cap, rate changing mid-year, missing
rate, and UQC mismatch.
- Changing a rate row's effective date changes only the shipments in the affected range.
- A rounding test proves the sum of item amounts equals the bill total.
```

Prompt 9: Leakage dashboard and deadlines

PROMPT 9 - DASHBOARD AND DEADLINES

```
GOAL
The main screen: money found, money at risk, and deadlines, readable in 5 seconds on a phone.

DO
1. Top row of Stat cards with count-up animation: "Unclaimed incentive" (sum of missing + short +
not_claimed), "At risk (deadline in under 60 days)", "Matched and closed", "Scrips expiring in 90
days". Each is tappable and opens a filtered list.
2. Realisation deadline engine: realisation_due_date = shipment date + realisation period from
compliance_rules (effective on that date; default 9 months for most exports from 1 Oct 2026;
INR-invoiced exports use their separate period). Never hard-code. Show days left with a ProgressRing
and colour (green > 90 days, amber 30 to 90, red < 30, dark red overdue).
3. Shipments list: columns shipping bill, date, buyer, FOB, expected incentive, credited, payment
matched, days to deadline, status chips. Filters, search, sort, saved views. Cards on mobile with the
most important facts first.
4. Shipment detail page: timeline of its life (filed, cleared, incentive credited, payment received,
eBRC ready) with what is missing highlighted, and a "What to do next" box in plain language.
```

```
5. Alerts centre listing every open alert with severity, due date, one-tap resolve or snooze.
6. Consultant home: a table of all client IECs with leakage totals and next deadline, sortable, with one-tap switch into a client.
7. Export: "Download leakage report" as PDF and Excel. PDF has a clean cover, summary, per-shipment table and the footer "Prepared with ExportLeak". Excel has one sheet per issue type.
8. Empty and loading states for every widget; error boundaries; never show NaN or negative zero.

ACCEPTANCE CRITERIA
- Dashboard numbers equal sums computed independently in a test.
- Changing the realisation period in compliance_rules updates deadlines without redeploying.
- Lighthouse performance >= 85 on mobile for the dashboard with 500 shipments.
```

Prompt 10: The bulk eBRC Excel generator

PROMPT 10 - BULK eBRC EXPORT

```
GOAL
Generate the Excel file that DGFT's Bulk Generate eBRC feature accepts, from confirmed matches. This is the single most valuable feature. Accuracy matters more than speed.

CONTEXT
/fixtures/dgft-bulk-ebrc-sample.xlsx is the official sample template downloaded from the DGFT portal, and /fixtures/dgft-bulk-ebrc-rules.md contains the rules I copied from DGFT's user guide. Read both fully. If anything is missing or ambiguous (column order, date format, allowed values, port-code list, file size limit), STOP and ask me. Do not guess.

DO
1. Build the generator from the sample file's exact column structure. Write each value in the format the sample shows. Keep the sample's sheet name and header text exactly.
2. Validator (also usable standalone for the free public tool in Prompt 17): checks every row against the rules (required fields, date format, port code validity from a data file, currency consistency, amounts not above the IRM balance, same-currency clubbing only, file size under the limit). Output a list of row-level errors with plain-language fixes and an "auto-fix" for safe cases (for example date formatting).
3. Export screen: choose IRMs or a date range, preview rows, see validation results, fix inline, then download. Show a final confirmation: "Review this file before uploading it on the DGFT portal. You submit it yourself."
4. Version stamp inside the app (not in the Excel) recording which sample-template version the generator follows, plus a banner that tells admins to re-check the template quarterly.
5. Gate by plan: Free users can validate but not generate; Exporter and Consultant plans can generate.

ACCEPTANCE CRITERIA
- A round-trip test: generate a file from fixtures, re-read it, compare cell by cell with the sample's structure.
- Validator unit tests for each rule, each with a failing and a passing row.
- File never contains data from another IEC or organization (RLS test).
```

Phase C: Plans, trial and payments

Prompt 11: Plans, entitlements and usage metering

PROMPT 11 - ENTITLEMENTS

```
GOAL
One source of truth for what each plan can do, enforced on the server.

DO
1. /config/plans.ts defines plan keys: free, trial_pro, exporter_monthly, exporter_annual, consultant_monthly, consultant_annual. For each: display name, price in paise, billing period, Razorpay plan id env var name, and entitlements: max_iecs, shipping_bills_per_month, can_generate_ebrc, can_export_reports, alerts_channels, team_seats, retention_days.
2. A server function getEntitlements(orgId) that returns the effective entitlements from the organization's state: active subscription, active trial, grace period, or free fallback.
```

3. A function assertCan(orgId, action, amount) used inside every route handler and server action that is gated (upload, add IEC, generate eBRC, invite seat, export report). It throws a typed PlanLimitError that the UI turns into a helpful upgrade dialog showing exactly which limit was hit and which plan fixes it.
4. usage_counters increments inside the same database transaction as the action it counts. Monthly period boundaries in Asia/Kolkata. A counter is never decremented when a document is deleted (to avoid abuse).
5. Usage meter in the UI: "34 of 50 shipping bills used this month" with a progress bar that turns amber at 80% and red at 100%.
6. Never block read access to existing data when a plan lapses: users can always view, and export their own data. Only creation and premium features are gated.

ACCEPTANCE CRITERIA
- Tests for each plan: the boundary (limit-1, limit, limit+1) for each gated action.
- Calling a gated API directly with curl while on Free returns 402 with a machine-readable error code.
- Downgrade from Consultant to Exporter with 12 IECs keeps all 12 readable but blocks adding more.

Prompt 12: Trial engine

PROMPT 12 - 7-DAY TRIAL WITHOUT A CARD

GOAL
Let users try the full Exporter experience for 7 days without entering payment details, which builds trust, and then land gently on the Free plan.

DO
1. startTrial(orgId): allowed once per organization and once per IEC code across the whole system (prevent repeat trials by creating new accounts); sets trial_ends_at = now + 7 days and plan_key = trial_pro. Idempotent.
2. A daily scheduled job: at trial end, set plan_key = free. No data is deleted; premium features lock; existing data stays readable.
3. In-app countdown banner: "Pro trial: 5 days left" with a primary "Choose a plan" button. Day 5 and day 7 emails (and an optional WhatsApp reminder template stored for later): friendly, shows what they found during the trial ("You have ₹X in unclaimed incentives flagged") and the single upgrade link.
4. If a user subscribes during the trial, the trial ends immediately and the paid plan starts (Prompt 14 sets the state).
5. Admin override: extend a trial by N days for a specific organization (audit-logged). Useful for pilots with consultants.
6. Events for analytics: trial_started, trial_day_5_email, trial_expired, trial_converted.

NOTE ON CARD-FIRST TRIALS
Razorpay Subscriptions can start billing at a future date, which allows card-first trials. Do NOT implement that now. Add a short design note file explaining how it could be done later, and mark it clearly as unverified against Razorpay's current documentation.

ACCEPTANCE CRITERIA
- Time-travel test with a fake clock: day 0 start, day 5 reminder queued, day 7 downgrade, data still readable.
- A second trial for the same IEC code is refused with a friendly message.

Prompt 13: Razorpay checkout and subscription creation

PROMPT 13 - RAZORPAY CHECKOUT

GOAL
Let a user choose a plan, pay on Razorpay's checkout, and be ready for the webhook to activate the plan.

SETUP FACTS (Razorpay test mode first)
Plans for exporter_monthly, exporter_annual, consultant_monthly, consultant_annual are created in the Razorpay dashboard or via API; their ids are in environment variables. Keys: RAZORPAY_KEY_ID (public), RAZORPAY_KEY_SECRET (server only), RAZORPAY_WEBHOOK_SECRET (server only, separate from the key secret).

```
DO
1. Route POST /api/billing/subscribe { planKey }:
   - requireOrgAccess(owner or admin). Validate planKey against config.
   - Create or reuse the Razorpay customer for the org (store razorpay_customer_id).
   - Create a Razorpay subscription with plan_id, a sensible total_count (for example 120 for monthly,
10 for annual), customer_notify = 1, and notes { org_id, plan_key }.
   - Insert a subscriptions row with status "created". Return { subscriptionId, keyId }.
2. Client: load checkout.js on demand, open Razorpay Checkout with the subscription_id, prefill name,
email and contact, theme colour from our tokens, and the app name and logo. UPI Autopay and cards
should appear as Razorpay enables them on the account; do not assume which methods are available.
3. On the checkout "handler" callback the browser receives razorpay_payment_id,
razorpay_subscription_id and razorpay_signature. Send them to POST /api/billing/verify which verifies
the signature server-side (HMAC-SHA256 over "payment_id|subscription_id" with the key secret; note
this order differs from one-time orders). This result is used ONLY to update the UI message. It never
activates the plan.
4. After the handler fires, show a full-screen "Confirming your payment..." state (animated), then
subscribe to realtime changes (or poll GET /api/billing/status every 2 seconds for up to 90 seconds).
When the webhook activates the plan (Prompt 14) the UI shows a success animation and unlocks features
instantly. If 90 seconds pass, show "Payment received, activation is taking longer than usual" with a
support button and keep polling gently.
5. Handle checkout dismissal, payment failure, and "already subscribed" (block a second active
subscription for the same org; offer plan change instead).
6. Plan change in v1: to upgrade, create the new subscription, and cancel the old one at cycle end (or
immediately after the new one is active). Keep it simple and document the behaviour in the UI ("Your
new plan starts today; your old plan will not renew").
7. Cancellation: POST /api/billing/cancel cancels at cycle end; the user keeps access until
current_end.

ACCEPTANCE CRITERIA
- Using Razorpay test cards/UPI, a full purchase works end to end (after Prompt 14 is also done).
- The browser callback alone, with the webhook blocked, does NOT activate the plan (test this).
- Secrets never reach client bundles (grep the build output in a test).
```

Prompt 14: Razorpay webhooks (the activation engine)

PROMPT 14 - WEBHOOKS

```
GOAL
A secure, idempotent webhook endpoint that is the single source of truth for activating, renewing,
downgrading and cancelling plans.

DO
1. Route POST /api/webhooks/razorpay:
   - Read the RAW request body (do not parse JSON before verifying; use request.text()).
   - Verify the X-Razorpay-Signature header: HMAC-SHA256 of the raw body with RAZORPAY_WEBHOOK_SECRET,
hex encoded, compared with timingSafeEqual. If the secret is missing or the signature is invalid,
return 400 and process nothing (fail closed).
   - Parse JSON, extract the event id (use Razorpay's event id header if present, otherwise a SHA-256
of the raw body) and insert into payment_events with a UNIQUE constraint. If the insert conflicts,
return 200 immediately (already processed).
   - Process inside a database transaction, then mark processed_at. Return 200 quickly; move slow work
(emails) to a queue.
   - If processing fails, store the error on the payment_events row and return 500 so Razorpay
retries.
2. Handle these events (look up the org via notes.org_id, falling back to razorpay_subscription_id):
   - subscription.authenticated: mark pending.
   - subscription.activated and subscription.charged: set status active, plan_key from notes,
current_start/current_end/charge_at/paid_count from the subscription entity; set organization
plan_key and plan_status = active; end any trial; clear alerts about payment problems; send a receipt
email.
   - subscription.pending: payment retry in progress; set plan_status = past_due and start a 3-day
grace period (config value) during which features stay on and a banner appears.
   - subscription.halted: retries exhausted; downgrade to free after grace.
```

```
   - subscription.cancelled: if cancel_at_cycle_end, keep access until current_end then downgrade;
else downgrade now.
   - subscription.paused / resumed / completed / updated: map to states sensibly and document in a
state diagram file.
   - payment.failed: record, notify the owner with a link to update the payment method.
   - refund.created / processed / failed: record and, for processed refunds of the latest charge,
downgrade if refund policy says so.
3. Out-of-order and duplicate safety: never move a subscription backwards because of an older event
(compare entity timestamps/status ranks). Reprocessing the same event must be a no-op.
4. Reconciliation job (daily): for every non-final subscription, fetch its current state from
Razorpay's API and fix any drift; log differences.
5. Admin page /admin/billing: recent webhook events with status, payload viewer (secrets masked), a
"reprocess" button, and search by subscription id.
6. Local testing: a script that sends signed sample payloads to the endpoint (using the webhook
secret) for each event type and for duplicates, invalid signatures and out-of-order sequences.
Document how to use Razorpay's dashboard to point the webhook at a tunnel URL in test mode.

SECURITY NOTES
Never log full payloads with personal data. Never accept a webhook without signature verification. The
browser callback from Prompt 13 must not call any function used here.

ACCEPTANCE CRITERIA
- Tests: valid signature accepted; one flipped byte rejected; same event twice processed once;
activated then older authenticated event does not downgrade; halted triggers downgrade after the grace
period; cancel at cycle end keeps access until current_end.
- End to end in test mode: pay, see the plan unlock within seconds without a page refresh.
```

Prompt 15: Billing UI, pricing page and legal pages

PROMPT 15 - BILLING UI AND LEGAL PAGES

```
GOAL
A clear pricing experience and a billing area that builds trust. Razorpay's account activation also
requires proper public policy pages.

DO
1. Pricing page (public and in-app): three cards (Free, Exporter, Consultant) with a monthly/annual
toggle (animated), a "most popular" badge on the plan we want users to choose, plain-language feature
lists, an FAQ ("Do you need my DSC? Never." "Can I cancel? Yes, any time." "What happens to my data if
I stop paying? You keep read access and can export it."), and a comparison table that collapses into
accordions on mobile.
2. Upgrade dialogs from PlanLimitError (Prompt 11) deep-link to this pricing view with the relevant
plan highlighted.
3. Billing page: current plan, status, next charge date and amount, payment method hint if available,
usage meter, buttons: change plan, cancel at cycle end, resume if cancelled-but-not-ended. Show
billing history (from payment events or Razorpay payments). Receipts: simple branded receipt PDFs now;
a proper GST invoice generator is a later task and must wait until a CA confirms our GST status. Do
not claim to issue GST invoices before that.
4. Dunning UI: banners for past_due and grace states with a one-tap path to fix payment.
5. Public pages with real content placeholders I will edit: Terms of Service, Privacy Policy (mention
AI processing of uploaded documents, retention, deletion rights, India DPDP considerations in plain
language), Refund and Cancellation Policy (draft a fair policy: for example a full refund within 7
days of the first paid charge), Contact page with email and WhatsApp, About page with the founder's
name and photo placeholder (trust).
6. Footer on every public page linking to all of these plus company details placeholders.

ACCEPTANCE CRITERIA
- All pages work at 360px and have no placeholder lorem ipsum in navigation or buttons.
- Toggling annual/monthly updates prices with a smooth animation and correct discount badge.
- A lapsed user can still open Billing and Export my data.
```

Phase D: Growth, polish and launch

Prompt 16: Alerts and notifications

PROMPT 16 - ALERTS

```
GOAL
Proactive alerts so users get value without logging in.

DO
1. Alert generator (daily scheduled job per IEC): realisation deadline approaching (90, 60, 30, 7 days), overdue, scrip expiring (90, 30, 7 days), shipping bills missing a payment link, unclaimed incentive found, trial reminders, payment problems.
2. Deduplicate: one alert per (kind, entity, threshold). Resolve automatically when the cause disappears.
3. Channels: in-app (bell with unread badge), email via Resend with a weekly digest option, and a WhatsApp-ready message template stored in the database for later integration (do NOT integrate WhatsApp yet; verify WhatsApp Business API rules and costs first).
4. Notification preferences page: per alert type, per channel, digest frequency, quiet hours. Consultants can choose per client.
5. Every email has a one-click unsubscribe link and a clear sender name.

ACCEPTANCE CRITERIA
- Fake-clock test: deadline thresholds produce exactly one alert each.
- Unsubscribed users receive nothing except essential billing messages.
```

Prompt 17: Landing page, free tools and SEO

PROMPT 17 - LANDING PAGE, FREE TOOLS, SEO

```
GOAL
Zero-budget marketing engine: a landing page that converts and free public tools that attract exporters through Google search.

DO
1. Landing page (fast, mobile-first, strong copy placeholders): hero with the promise "Find the export incentive money you are missing", a 15-second looping product demo (build as an animated component using the demo data; no fake screenshots of government sites), "How it works" in 3 steps, the trust section (never asks for DSC or passwords, files private, delete anytime), pricing, FAQ, and a founder note. Sticky mobile CTA "Check my shipments free".
2. Free tool 1: RoDTEP and drawback calculator. Input HS code, FOB value, quantity and unit; output expected incentive using the rate tables, with the formula shown. Shows a clear "rates last updated on <date>" and a disclaimer. No signup required.
3. Free tool 2: Bulk eBRC Excel validator. User drops their own Excel; the validator from Prompt 10 lists errors and offers a corrected file. Process in memory, store nothing, and say so on the page. Rate-limit per IP.
4. Free tool 3 (optional): Shipping Bill PDF to Excel extractor for up to 5 PDFs per day (uses the Prompt 6 parser; strict spend cap; show a signup nudge to unlock reconciliation).
5. Programmatic SEO: a /learn glossary with one page per term from the Beginner Guide (RoDTEP, eBRC, IRM, EDPMS, drawback, scrip, IEC and so on), each with a plain-language explanation, a "common mistakes" section and a link to the relevant tool. Content must be accurate; mark pages "last reviewed" with a date and keep a reviewer-note field.
6. Technical SEO: semantic HTML, unique titles and meta descriptions, canonical URLs, sitemap.xml, robots.txt, Open Graph images generated per page, JSON-LD (Organization, FAQ, SoftwareApplication), fast loading (no layout shift), image optimisation.
7. Analytics: privacy-friendly, cookie-light (events: tool_used, signup_started, signup_completed, pricing_viewed).
8. Share loop: after a user generates a report, add an optional "Share a summary with your CA" link that creates a read-only, expiring, revocable page with only the totals (no document download). The footer says "Prepared with ExportLeak".

ACCEPTANCE CRITERIA
- Lighthouse mobile: Performance >= 90, SEO >= 95, Accessibility >= 95 on landing and tool pages.
```

```
- Free tools work without login and never persist uploaded files (test checks storage is empty afterwards).
- Rate limiting returns a friendly message, not a crash.
```

Prompt 18: Mobile, motion and accessibility polish

PROMPT 18 - POLISH

```
GOAL
Make the whole app feel fast, calm and premium on a mid-range Android phone as well as on desktop.

DO
1. Review every screen at 360, 390, 768, 1024 and 1440 px. Fix overflow, cramped tap targets, tiny text (body >= 16px on mobile), sticky headers hiding content, and modal sizing. Use bottom sheets for forms on mobile.
2. Motion pass: consistent, subtle, purposeful. Page transitions, shared-element feel for opening a shipment from the list, skeleton loading for every async region, success moments (a soft check animation after upload, after matching, after payment), animated progress rings and count-up numbers. Keep animations under 300ms except count-ups. Provide a reduced-motion alternative for everything.
3. PWA: manifest, icons, theme colour, install prompt (shown after the second visit), offline shell with a friendly offline page, and safe handling of uploads that fail offline (retry on reconnect). Do not cache private data in the service worker.
4. Performance budget: initial JS under 200 KB gzip on the landing page, lazy-load PDF viewer, charts and Razorpay script, virtualise long lists, optimise images, use the Next.js font loader.
5. Accessibility: WCAG 2.1 AA colour contrast, visible focus rings, labels on all inputs, aria-live announcements for upload and payment status, full keyboard navigation, screen-reader-friendly tables, semantic landmarks.
6. Microcopy pass: plain, friendly English; Indian number formats; no jargon without a tooltip (RoDTEP, IRM, eBRC each get a hover/tap explanation linking to the glossary).
7. Optional: an i18n structure ready for Hindi later (do not translate now).

ACCEPTANCE CRITERIA
- Playwright screenshots at 4 viewports for the 10 key screens, committed for review.
- axe accessibility tests pass with zero serious violations.
- Throttled "Slow 4G, 4x CPU" run keeps the dashboard interactive in under 4 seconds.
```

Prompt 19: Security, privacy, testing and deployment

PROMPT 19 - HARDENING AND LAUNCH

```
GOAL
Make the app safe to put in front of paying business customers.

DO
1. Security review of the whole codebase against this checklist and fix everything found: authentication on every route, RLS tests for every table, input validation with Zod, output encoding, CSRF protection for state-changing routes, rate limiting on login, upload and free tools, secure headers (CSP, HSTS, frame-ancestors, referrer policy), secrets only in environment variables, no secrets in client bundles, dependency audit, signed URLs with short expiry for downloads, malware-safe file handling.
2. Privacy features: export all my data (zip), delete my account and all data (with a confirmation step and a 30-day purge for backups), data retention settings, an audit log viewer for owners, and a plain-language "How we handle your documents" page. Avoid collecting anything beyond what the product needs. Keep a short DPDP checklist file in the repo with assumptions to confirm with a lawyer.
3. Testing: unit tests for the matcher, calculator, validator, entitlements and webhook handler; integration tests with a local database; Playwright end-to-end tests for: sign up, onboarding, upload and parse (with fixtures), match review, dashboard, eBRC export, upgrade with a mocked Razorpay, webhook activation, cancel, trial expiry. Configure CI to run all of them on every push.
4. Observability: Sentry for errors, structured logs without personal data, health check route, uptime monitor, and an admin dashboard with signups, trials, conversions, MRR (computed from active subscriptions) and failed webhooks.
5. Backups and recovery: enable database backups, document the restore procedure, test restoring once.
```

```
6. Deployment: production and staging environments with separate Supabase projects and separate
Razorpay keys (test for staging, live for production), environment variable documentation, migration
process, a rollback plan, a custom domain with HTTPS, and a launch checklist file.
7. Cost guardrails: alerts when AI parsing spend, storage or email volume cross thresholds that would
break the monthly tools budget.

ACCEPTANCE CRITERIA
- CI green. A written security review report (what was checked, what was fixed, what remains) is
committed.
- Restoring a backup into staging works and the app boots against it.
- A staged "live-like" run: payments in test mode, emails delivered, webhook retries visible, no
errors in Sentry.
```

Appendix A: Razorpay go-live checklist

- **Business setup:** decide the legal form and bank account for receiving payments; complete Razorpay KYC and activation. Ask a CA about GST registration and invoicing before charging customers.
- **Public pages live:** pricing, Terms, Privacy, Refund and Cancellation, Contact (email and phone), About. Razorpay reviews these during activation.
- **Plans created in live mode:** same names and amounts as in test mode; copy the live plan ids into production environment variables.
- **Webhook:** configure the production URL (HTTPS) in the Razorpay dashboard with the secret you chose; select the subscription, payment and refund events your handler supports; keep test and live webhooks and secrets separate.
- **Keys:** live key id and secret only in the production environment; rotate if ever exposed.
- **Payment methods:** confirm which methods (cards, UPI Autopay, netbanking mandates) are enabled on your account and test each one with small real payments before announcing.
- **First real payment:** pay for your own plan with your own card, confirm activation by webhook, cancel, and confirm the refund and downgrade behaviour match your policy.
- **Monitoring:** watch webhook failures and payment failures daily in the first two weeks.

Appendix B: Review prompts to run after building

REVIEW PROMPT 1 - ADVERSARIAL SECURITY REVIEW

```
Act as a hostile penetration tester and a payments auditor. Review this entire repository. Try to: (1)
read another organization's data through every route and server action; (2) activate a paid plan
without a valid webhook; (3) replay or forge a Razorpay webhook; (4) bypass plan limits by calling
APIs directly; (5) upload a malicious file; (6) leak secrets into the client bundle; (7) cause double
billing or a stuck subscription state. For each finding give severity, the exact file and line, a
reproduction, and a fix. Then implement fixes for all High and Critical findings and add a regression
test for each.
```

REVIEW PROMPT 2 - CORRECTNESS REVIEW OF MONEY LOGIC

```
Review the matching engine, incentive calculator, deadline engine and eBRC generator purely for
correctness. List every assumption each one makes. For each assumption say whether it comes from a
fixture/official data, from me, or from your guess. Produce a table of every place where a guess
exists and add a visible warning or a blocking error in the product for each. Write 10 additional
adversarial test cases per module (rounding, leap years, month-end deadlines, currency edge cases,
duplicate invoice numbers) and fix any bugs found.
```

REVIEW PROMPT 3 - FIRST-TIME USER REVIEW

```
Pretend you are a 50-year-old accounts manager at a small garment exporter in Tiruppur using a mid-range Android phone on a slow connection. Walk through the app from the landing page to the first leakage result and then to paying. List every moment of confusion, jargon, slowness, or distrust, with screenshots at 360px width, and fix the top 15. Keep the interface in plain English and make sure the "we never need your DSC or passwords" message appears at every point where a user might worry.
```

Appendix C: Things only you can do

1. Download the latest **bulk eBRC sample template** and the eBRC user guide from the DGFT portal and place them in /fixtures.
2. Obtain official **RoDTEP and drawback rate notifications** (as CSV or tables) for the products you want to support first, and prepare the rate CSV for Prompt 8. Start with one or two product families, not everything.
3. Collect 10+ **real, anonymised Shipping Bill PDFs**, an IRM export and a scroll export from friendly consultants. Get written permission and remove or mask names where possible.
4. Ask a CA about GST registration, invoicing and the right business structure before taking money.
5. Have a DGFT consultant review the validation rules and the calculation logic once; their feedback will be worth more than any amount of extra coding.
6. Run your outreach in parallel with the build: 10 to 15 personal messages per day to EXIM consultants. The prompts above build the product; they do not find customers.

This pack is a building aid, not legal, tax or compliance advice. Regulations (including the FEMA realisation period and DGFT file formats) have changed several times recently, so re-verify them before launch and every quarter after.